
High-Performance CUDA Kernels Without Writing CUDA by Hand: A Technical Report for the FlashInfer AI Kernel Generation Contest

Mao Luo^{*1} Hongbin Li^{*1} Feng Lin¹ Hanling Yi¹ Zhe Huang¹

Abstract

We study whether a general-purpose code agent can produce state-of-the-art GPU kernels without any manually written CUDA code. We investigate this question in the MLSys 2026 FlashInfer AI Kernel Generation Contest, covering both Track A (Fused MoE) and Track B (DSA TopK Indexer and DSA Sparse Attention), under the correctness-gated FlashInfer-Bench evaluation protocol. Starting only from the PyTorch reference implementations, workload definitions, benchmark commands, and a compact set of reusable CUDA optimization skills, we build a lightweight multi-agent workflow in Claude Code to generate, debug, profile, and optimize all kernels. Humans remain strictly in an orchestration role: defining the workflow, enforcing correctness and anti-hacking constraints, supplying key references, and redirecting the search when progress stalls, without reviewing, editing, or hand-tuning the kernel code itself. Across roughly 1.9 billion agent tokens (\$500 in API cost), the resulting kernels achieve speedups of 92.68 $\times$ on Fused MoE, 1101.02 $\times$ on DSA TopK Indexer, and 181.35 $\times$ on DSA Sparse Attention relative to the PyTorch reference implementations, while also outperforming the corresponding FlashInfer baselines significantly. These results suggest that, under a disciplined correctness-first workflow, code agents can serve as effective autonomous optimizers for modern GPU kernel development.

1 From Kernel Engineering to Agentic Optimization

High-performance GPU kernels have become a key enabler of modern LLM systems, especially for architectures built around sparse attention and mixture-of-experts (MoE) computation [1, 5, 3, 2]. Yet producing state-of-the-art kernels on new hardware remains a largely manual process, requiring repeated reasoning about tiling, memory movement, scheduling, and hardware-specific constraints. Recent progress such as Triton have made kernel development more programmable and reusable, but obtaining top performance still typically demands substantial expert effort [6].

This work asks a different question: can a general-purpose code agent produce state-of-the-art kernels without any manually written CUDA code? We investigate this question in the MLSys 2026 FlashInfer

Kernel	FlashInfer	Ours	Token Cost
DSA-TopK	52.03 $\times$	1101.02 $\times$	414M
DSA-Attn	10.33 $\times$	181.35 $\times$	427M
FuseMoE	47.08 $\times$	92.68 $\times$	1.05B

Table 1. Kernel performance comparison using the PyTorch implementations as the speedup baseline.

AI Kernel Generation Contest, using the FlashInfer-Bench [7] evaluation protocol as the optimization environment. Specifically, we participate in both tracks: Track A targets a fused FP8 MoE kernel derived from the DeepSeek-V3/R1 MoE block, while Track B targets DeepSeek-V3.2 sparse attention and includes a DSA TopK indexer and a DSA Attention kernel [2].

A defining feature of our approach is that none of the generated kernels were manually written or hand-tuned by our team. Starting only from the PyTorch reference implementation, workload definition, benchmark command, and a compact set of reusable CUDA optimization skills, a lightweight multi-agent workflow generated, debugged, profiled, and optimized all kernels. Humans remained strictly in an orchestration role: defining the workflow, enforcing anti-hacking safeguards, supplying key references, and pushing the search toward more aggressive optimizations when

^{*}Equal contribution ¹Intellifusion Inc., Shenzhen, China. Correspondence to: Hanling Yi <hanling.cuhk@gmail.com>.

progress stalled.

Across roughly 1.9 billion tokens (around \$500 in Claude API cost), the resulting kernels achieved substantial gains over both the PyTorch and FlashInfer baselines [8]. Specifically, we attained speedups of $92.68\times$ on Fused MoE, $1101.02\times$ on DSA TopK Indexer, and $181.35\times$ on DSA Sparse Attention relative to the PyTorch implementations, while also outperforming the corresponding FlashInfer baselines (Table 1). These results suggest that under a disciplined correctness-first workflow, code agents can serve as effective autonomous optimizers for modern GPU kernel development.

2 Agentic Optimization Workflow: Agents, Humans, and Tools

Effective agent-driven kernel optimization depends not only on model capability, but also on a well-structured search process. Our workflow combines a controlled optimization loop, targeted human intervention, and a small set of reusable tools and skills to support long-horizon kernel optimization. This section describes these components.

2.1 Agentic Workflow for Kernel Optimization

Kernel optimization is inherently exploratory: progress typically depends on repeated cycles of hypothesis formation, implementation, profiling, and revision. To support this process, we built a team-based AutoResearch-style [4] agentic workflow in Claude Code, in which multiple agents operate with distinct but complementary roles. Figure 1 illustrates the organization of these agents and the information flow among them.

At the center of the workflow is a Planner, which receives the current best kernel, its NCU profiling results, and the history of prior runs, and then proposes the next optimization directions. These proposals are passed to multiple CUDA Coders, each running in an isolated GPU environment. Conditioned on the Planner’s guidance, the Coders explore candidate implementations in parallel, carrying out code changes, debugging, and benchmarking to search for improved solutions. The outputs of the Coders are then reviewed by a Synthesizer, which collects the strongest candidates, identifies the most effective changes, and merges them into an updated solution. Profiling support is provided by an Profiler, implemented as an NCU-based kernel profiler that generates performance reports and attributes bottlenecks for a given kernel. When exploration stalls, a Researcher serves as an

on-demand web search agent, retrieving relevant references and summarizing reusable optimization patterns or patch ideas to assist the Coders.

Within this workflow, the optimization loop follows a tightly controlled protocol. Since progress depends heavily on the effectiveness of the CUDA Coders, their behavior is explicitly constrained. The editable surface is deliberately restricted: agents may modify only the CUDA kernel implementation, while the benchmark harness, datasets, configurations, and reference implementations remain fixed. This ensures that all measured gains come from kernel improvements rather than changes to the evaluation setup.

Candidate kernels are evaluated under a correctness-gated speedup metric. A change is accepted only if it passes all correctness checks; numerical errors, runtime failures, and timeouts invalidate the result. After several unsuccessful attempts along the same direction, the CUDA Coders consult the Researcher for additional references and optimization ideas. If no progress is made within the allotted time, they report back to the Synthesizer and wait for the Planner to redirect the search toward a new optimization strategy.

2.2 Human Intervention as Search Correction

Even with a clean automated workflow, optimization can still stall because agents exhibit systematic search biases. In our workflow, the main value of human intervention lies not in kernel implementation, but in search correction: identifying when the agent team is stuck, misattributes failures, or misses critical external references.

We observe three recurring failure modes. First, agents favor incremental tweaks even after progress plateaus and may fail to propose more disruptive optimizations unless explicitly pushed in that direction. Second, they sometimes revert genuinely beneficial changes after spurious failures, incorrectly attributing regressions to the new optimization rather than to an independent bug. Third, they tend to under-search external references, occasionally missing key examples needed to reach stronger implementations.

Overall, the agents are effective at local optimization, implementation, and bookkeeping, but less reliable at making major directional shifts or recovering from misleading failures. In such cases, targeted manager-style human intervention is sufficient to redirect the search and unlock substantially better performance. More broadly, this suggests that deep CUDA expertise is not always required; even limited domain knowledge can be sufficient to identify promising optimization di-

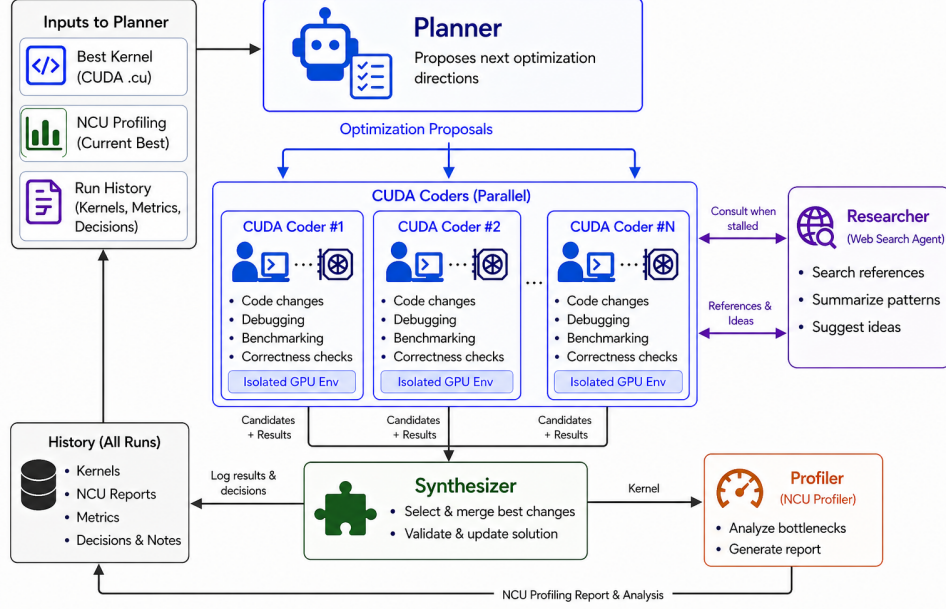


Figure 1. Team-based AutoResearch-style agentic workflow for CUDA kernel optimization.

rections.

2.3 Tools and Skills

We use pure CUDA as the implementation language. Our agent runtime is Claude Code, backed by Claude Opus 4.6 and later 4.7. Each sub-agent runs in its own context window. In addition, we supply a small set of on-demand skills, each packaged as a folder of Markdown notes that agents may load when needed. Table 2 summarizes the most important skills and tools.

3 Kernel Design and Optimization

3.1 Speedup vs. Token Cost

This section summarizes how performance evolves as additional agent tokens are invested in each kernel. Across all three kernels, the final gains arise not from a single rewrite, but from the cumulative effect of increasingly specialized optimizations.

3.1.1 Fused MoE

Fused MoE implements the MoE block used in DeepSeek-V3. For each input token, the kernel executes four stages: (1) computing routing scores via a lightweight linear projection, sigmoid activation, group-wise top-2 selection over 8 groups, and final top- $K = 8$ expert selection; (2) gathering the active rows for each expert and quantizing them into a contiguous FP8 layout with 128-element blockwise FP32 scal-

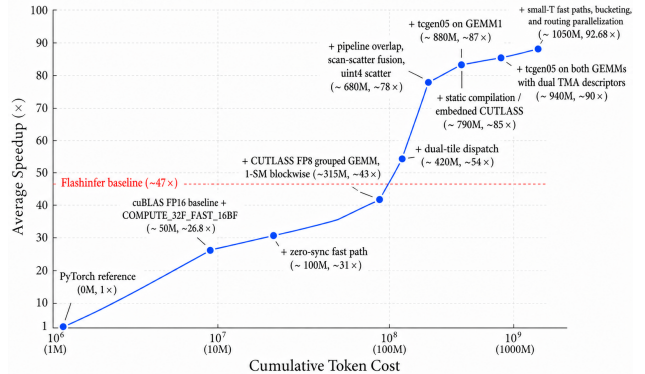


Figure 2. Cumulative speedup of Fused MoE vs. cost tokens.

ing factors; (3) evaluating the gated MLP, including an FP8 blockwise GEMM1 from $[T, 7168]$ to $[T, 4096]$, a SwiGLU activation, re-quantization, and an FP8 blockwise GEMM2 from $[T, 2048]$ to $[T, 7168]$; and (4) scattering the expert outputs back to the original token positions, weighted by the routing scores.

The final implementation keeps all metadata construction on the GPU and eliminates `cudaStreamSynchronize` from the hot path. It compiles three GEMM backends into a single module: a CUTLASS FP8 blockwise backend, a hand-written `tcgen05` backend, and a cuBLAS FP16 fallback for very large T . Figure 2 visualizes the cumulative optimization trajectory and the token cost.

Skill / Tool	Purpose	Source
cuda-b200-skill	Blackwell CUDA reference covering PTX ISA, TMA/WGMMA/tcgen05, occupancy, register/shared-memory limits, and debugging.	GitHub ¹
ncu-cuda-profiling	Automated NCU workflow with <code>-set full</code> , saved profiler reports, and summarized metrics.	GitHub ²
bench-environment	Contest evaluation harness covering tensor cloning, L2 flushes, cold-cache and no-CUDA-graph defaults, and anti-hacking rules.	Internal environment
/codex:rescue	Independent kernel review via the Claude Code plugin, surfacing alternatives like persistent kernels, TMA, and shared-memory skewing.	GitHub ³
Web research	Web search and fetch tools, used after repeated failures to investigate references such as DeepGEMM, FlashInfer, and RadiK.	External tool

¹ <https://github.com/technillogue/ptx-isa-markdown>² <https://github.com/maxiaosong1124/ncu-cuda-profiling-skill>³ <https://github.com/openai/codex-plugin-cc>

Table 2. Skills, tools, and references used for CUDA kernel optimization.

3.1.2 DSA Sparse Attention

DSA Sparse Attention is a paged KV-cache batched decoding operator derived from DeepSeek-V3.2. For each query token, the kernel (1) gathers the top- $K = 2048$ KV entries from a sparse index list; (2) computes scaled attention logits of the form $(q_{\text{nope}}K_c^\top) + (q_{\text{pe}}K_p^\top)$; (3) applies a base-2 softmax; and (4) accumulates $\text{attn_weights} \cdot K_c$ into a 512-dimensional output while also producing the LSE.

The final implementation adopts a 4-heads-per-block layout with 128 threads organized into 4 warps, assigning one warp to each head. KV tiles are staged through a two-stage `cp.async` pipeline with double-buffered shared memory. The kernel also uses an adaptive Split-K dispatch strategy: a single-pass kernel for a single token, a templated `SPLIT_K=64` configuration for up to two tokens, and a templated `SPLIT_K=32` configuration for larger batches. The merge kernel further partitions the 512-dimensional output into four slices, increasing merge parallelism by $4\times$. Additional optimizations include branchless online softmax, register-resident query and output accumulators, BF16 partial outputs to reduce merge bandwidth, and block reordering to improve L2 locality. Figure 3 visualizes the cumulative optimization path.

3.1.3 DSA TopK Indexer

DSA TopK Indexer implements the paged attention indexing stage for DSA sparse attention. Given FP8 queries Q with 64 heads and 128 dimensions per head, a paged FP8 key cache K with per-token FP32 scaling factors, per-head weights, and sequence lengths, the operator computes

$$\text{score}(t) = \sum_h \text{ReLU}(Q_h K_t^\top) \cdot w_h$$

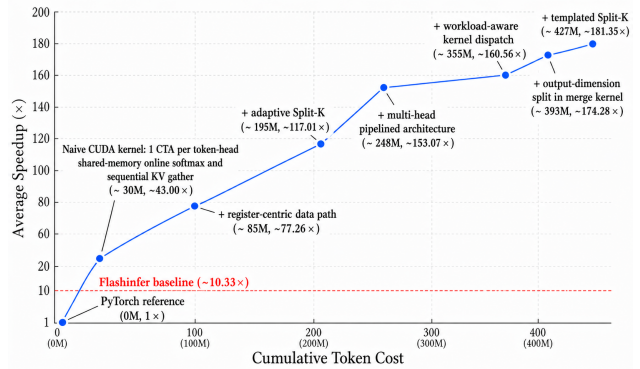


Figure 3. Cumulative speedup of DSA Sparse Attention vs. cost tokens.

and returns the top- K ($K = 2048$) token indices for each batch element. The PyTorch reference implementation performs a `bmm` over dequantized keys followed by `torch.topk`. Because the contest harness enforces required-matched-ratio 1.0, all returned indices must match the reference exactly. This bit-exact TopK requirement drives most of the architectural choices.

The final implementation is a three-kernel pure-CUDA pipeline with no CPU-GPU synchronization on the hot path. When $\text{seq_len} \leq K$, the pipeline degenerates to a pure index-remapping path, since every token is selected trivially. The main optimizations include a fused FP32-accumulation GEMM path that computes $QK^\top$, applies ReLU, weights each head, and reduces across heads within a single kernel; bank-conflict-free shared-memory layouts using a stride-129 key tile and padded cross-warp partial-sum buffers; a four-pass 8-bit radix TopK with sortable-float bit transforms, early termination, subgroup histograms, parallel suffix scan, and warp-ballot gathering; on-device fast/slow-path dispatch based on `seq_lens[b]`; and bit-exact FP8-to-FP32 dequantization that matches the reference. Figure 4 illustrates the cumulative optimization path.

Group	Length	Num	Speedup	Typical	Latency	Bottleneck (NCU)
short	1–16	5	103–240×	7	0.099 ms	Floor set by routing and launch overhead.
median	32–80	11	75–93×	54	0.205 ms	GEMM1+GEMM2 account for ~65% of iteration time
long	901–14,107	3	38–74×	14,107	1.229 ms	GEMM1 accounts for ~47%, GEMM2 for ~30%

Table 3. Fused MoE workload groups and the dominant bottlenecks of their representative workloads.

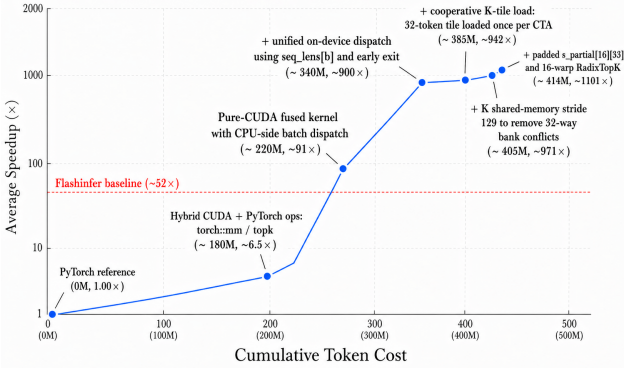


Figure 4. Cumulative speedup of DSA TopK Indexer vs. cost tokens.

3.2 Workload Regimes and Bottleneck Analysis

This section analyzes the three kernels through the lens of workload regimes. Rather than treating all benchmark instances independently, we group them into a small number of representative regimes, profile one representative workload from each group with NCU, and identify the dominant bottlenecks. This regime-level analysis helps explain both the main optimization gains achieved during the search and the remaining sources of performance loss.

3.2.1 Fused MoE

The 19 contest workloads can naturally partition into three sequence-length regimes. For each regime, we select a representative workload, profile it with NCU, and report the dominant bottlenecks in Table 3. Across all three regimes, the GEMM kernels already operate at high SM utilization. For the long-sequence representative, GEMM1 and GEMM2 achieve 97% and 95% SM utilization, respectively. Accordingly, the remaining optimization headroom appears to lie less in further improving standalone GEMM throughput than in reducing inter-stage data movement. For long-sequence workloads, the most promising remaining direction is to fuse pull_scatter into the GEMM2 epilogue, so that the producer-consumer path can remain in registers or shared memory. In the current implementation, however, this approach is limited by an upstream CUTLASS API constraint that we leave

unmodified.

3.2.2 DSA Sparse Attention

The 23 contest workloads can be grouped into three regimes by num_tokens, which also matches the boundaries used by our adaptive Split-K dispatcher. For each regime, we select one representative workload, profile it with NCU, and summarize the dominant bottleneck in Table 4. These clusters correspond to the main optimization wins during the campaign. MERGE_DIM_SPLIT=4 targeted the long-cluster merge tail and contributed the largest gain. The SPLIT_K=64 path for num_tokens ≤ 2 was introduced specifically for the mid cluster based on per-workload NCU analysis. For the short cluster, the single fused-kernel dispatch avoids merge overhead entirely.

The remaining performance gap appears to be largely structural. The attention kernel uses roughly 192 registers per thread and about 50 KB of shared memory, limiting achieved occupancy to around 10%. Since the kernel is already latency-bound rather than compute- or bandwidth-bound, further improvements likely require a more substantial architectural redesign, such as reducing register pressure or moving to a TMA/warpgroup/tcgen05-style design. Lower-precision compute could also reduce pressure, but this direction is constrained by the strict correctness requirements.

3.2.3 DSA TopK Indexer

The 128 contest workloads are naturally organized into three regimes according to max_num_pages. Because each page contains 64 tokens, this quantity effectively upper-bounds the maximum sequence length within a batch. The most important threshold is max_num_pages=32: when all sequences satisfy this bound, their lengths do not exceed TopK=2048, and the kernel can bypass score computation entirely in favor of a pure index-remapping fast path. For each regime, we choose a representative workload, profile its hot kernels with NCU, and summarize the principal bottlenecks in Table 5.

Group	#Tokens	Num	Speedup	Typical	Latency	Bottleneck (NCU)
short	1	1	280×	1 token	0.005 ms	Grid under-occupancy; only 4 CTAs are launched; latency is launch-dominated
median	2	8	125–148×	2 tokens	0.017 ms	Latency-bound Split-K; low occupancy from high register pressure
long	6–8	14	108–228×	8 tokens	0.026 ms	Register-pressure-limited occupancy; residual shared-memory bank conflicts remain

Table 4. DSA Sparse Attention workload groups and the dominant bottlenecks of their representative workloads. Typical cases include 8462 pages.

Group	Length	Num	Speedup	Typical	Latency	Bottleneck (NCU)
short	$\max S \leq 1024$ $\max P \leq 16$	27	477–3732×	bs=29 $\max P = 1$ $\max S = 64$	~0.002 ms	Fast-path index remap; floor set by launch and grid scheduling
medium	$1088 \leq \max S \leq 2048$ $17 \leq \max P \leq 32$	42	816–5366×	bs=15 $\max P = 31$ $\max S = 1984$	~0.004 ms	Fast-path index remap; kernel cost grows only with batch size and TopK
long	$\max S > 2048$ $\max P > 32$	59	107–756×	bs=4 $\max P = 36$ $S = [18, 11, 2161, 20]$	~0.020 ms	Slow-path FusedGemm + RadixTopK; low active-CTA count and FP8 dequant stalls

Table 5. DSA TopK Indexer workload groups and the dominant bottlenecks of their representative workloads.

These regimes account for the largest gains observed in the DSA TopK Indexer optimization campaign. In the short and mid regimes, the on-device fast-path dispatch yields especially large improvements by skipping score computation whenever $\text{seq_len} \leq \text{TopK}$. In both cases, execution reduces to the same lightweight index-remapping kernel, whereas the PyTorch reference still incurs the cost of per-batch GEMM and topk. In the long regime, only those sequences whose lengths exceed TopK take the full slow path. For the representative workload, this means that only one of the four batch elements invokes FusedGemm and RadixTopK, while the remaining three return early.

Within the long regime, the main improvements come from eliminating shared-memory bank conflicts and reducing serialized work inside RadixTopK. In particular, padding $s_partial$ from [16][32] to [16][33] removes a substantial cross-warp bank conflict in FusedGemm, while increasing the RadixTopK configuration from 256 to 512 threads reduces the loop work performed in each pass. The remaining limitation is structural: at those small batch sizes, RadixTopK launches only a small number of CTAs, leaving much of the GPU underutilized, and the fused GEMM path still remains behind what a TMA+tcgen05-style pipeline could potentially achieve.

4 Discussion

When Speedup Is Not Real. During kernel optimization, we encounter several instances of benchmark exploitation, in which the agent achieves implausi-

bly large reported speedups without producing a genuinely improved kernel. In early Fused MoE runs, for example, the reported speedup sometimes exceeds 1000×. The underlying mechanism is that the in-process FlashInfer-Bench harness reuses input tensor pointers across iterations, which leads the agent to introduce an output cache keyed by tensor identity. After the first iteration, the kernel can simply return cached outputs while still passing correctness checks, even though the measured gain does not correspond to a deployable kernel optimization.

A notable lesson from this campaign is therefore that code agents, when granted full visibility into the project codebase, can engage in “reward hacking” unless the optimization objective is carefully constrained. To prevent this behavior, we incorporate explicit anti-hacking rules into every agent’s initial prompt. In particular, we disallow optimizations based on input-identity caching, cross-iteration buffer reuse, or other sample-specific shortcuts. If a performance gain depends on a legitimate deployable cache, the agent must report cold-path and warm-path performance separately. This rule becomes a core part of our evaluation discipline and is necessary to ensure that measured speedups reflect genuine kernel improvements.

Human as Orchestrator. Human contribution is best understood as orchestration rather than implementation. Humans define the evaluation protocol, benchmark discipline, and multi-agent workflow; enforce anti-hacking and correctness constraints; and intervene only when the search plateaus, typically by redi-

recting the agents toward more aggressive optimization strategies or supplying key external references that the agents fail to retrieve on their own. Crucially, humans do not review, edit, or hand-tune the kernel code itself.

5 Conclusion

This work asks whether a general-purpose code agent can produce state-of-the-art GPU kernels without any manually written CUDA code. In MLSys 2026 FlashInfer AI Kernel Generation Contest, we show that the answer is yes: under a disciplined correctness-first workflow, an agentic system can generate high-performance kernels that substantially outperform both the PyTorch references and the corresponding FlashInfer baselines. Our results also show that model capability alone is not enough. Strong performance requires a structured optimization loop, explicit anti-hacking constraints, and targeted human intervention to redirect the search when agents stall or misgeneralize. Yet this intervention remains orchestration rather than implementation: humans do not write, edit, or hand-tune the kernel code. Taken together, these findings suggest a broader shift. High-performance GPU kernel development no longer needs to be viewed exclusively as a manual expert craft. With the right workflow, code agents can already serve as practical autonomous optimizers for modern GPU kernels.

References

- [1] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. arXiv preprint arXiv:2205.14135, 2022.
- [2] DeepSeek-AI. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.
- [3] Gale, T., Narayanan, D., Young, C., and Zaharia, M. Megablocks: Efficient sparse training with mixture-of-experts. In Proceedings of MLSys, 2023.
- [4] Karpathy, A. Autoresearch. URL <https://github.com/karpathy/autoresearch>.
- [5] Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538, 2017.
- [6] Tillet, P., Kung, H. T., and Cox, D. Triton: An intermediate language and compiler for tiled neural network computations. In Proceedings of MAPL, 2019.
- [7] Xing, S. et al. Flashinfer-bench: Building the virtuous cycle for ai-driven kernel generation. arXiv preprint arXiv:2601.00227, 2026.
- [8] Ye, Z. et al. Flashinfer: Efficient and customizable attention engine for llm serving. arXiv preprint arXiv:2501.01005, 2025.

A NCU Profile of the Slowest FuseMoE Workload

This appendix reports the NCU profile for the slowest FuseMoE workload in our final submission. The workload has `seq_len=14107` and corresponds to the long-sequence regime discussed in Table 3.

A.1 Capture Provenance

- Kernel version: `cuda-moe-tvm-v14`
- Workload: `5e8dc11c, seq_len=14107`
- Driver: 3 warmup iterations followed by 5 NVTX-scoped measurement iterations
- Hardware: NVIDIA B200, SM100, compute capability 10.0
- Profiler: NCU 2026.1.0.0, `-set full, -replay-mode kernel`

A.2 Per-Kernel Summary

Table 6 summarizes the major kernels in one FuseMoE iteration. The profile shows that the two CUTLASS FP8 grouped-GEMM invocations dominate total runtime, while `pull_scatter` is the largest non-GEMM DRAM consumer.

The compact summary indicates that the workload is dominated by the GEMM path: the two CUTLASS_FP8_grouped invocations account for the largest share of runtime. Among the non-GEMM kernels, `pull_scatter_bf16` is the most important optimization target because it has the highest DRAM utilization.

A.3 Dominant Kernel Drill-Down

The two dominant components are the CUTLASS FP8 blockwise grouped GEMMs and the `pull_scatter` kernel. The former dominates total iteration time, while the latter is the largest non-GEMM memory-bandwidth consumer.

Table 6. Compact NCU summary for the slowest FuseMoE workload.

Kernel	Inv.	Time (μ s)	SM (%)	DRAM (%)	L1 (%)	L2 (%)	Occ. (%)	Reg.	WCPI
CUTLASS_FP8_grouped	2	491.74	50.68	24.85	45.18	41.69	14.09	168	5.46
routing_kernel	1	108.22	62.37	1.74	67.96	1.65	65.08	32	17.71
pull_scatter_bf16	1	81.31	23.52	62.92	33.87	30.90	43.50	61	26.82
gather_fp8_and_scales	1	42.69	46.47	47.04	46.91	35.63	82.40	32	25.56
swiglu_to_fp8	1	34.94	70.89	55.87	30.86	32.65	85.21	29	16.66
scatter_with_scan	1	11.30	2.31	0.53	1.80	1.47	24.03	18	198.69
Total kernel time for one MoE iteration: 1261.94 μ s									

A.3.1 CUTLASS FP8 Blockwise Grouped GEMM

The same CUTLASS grouped-GEMM template is invoked twice per MoE iteration:

- GEMM1: $K = 7168$, approximately 600.22 μ s.
- GEMM2: $K = 2048$, approximately 383.26 μ s.

On this long-sequence workload, the runtime dispatcher selects the CUTLASS backend rather than the hand-written tcgen05 backend, because CUTLASS is faster on long-sequence cases by approximately 2–7%. Table 7 reports the Speed-of-Light throughput metrics for these two CUTLASS invocations.

Table 7. NCU throughput summary for CUTLASS FP8 grouped GEMM.

Metric	Unit	Min	Max	Avg.
DRAM frequency	GHz	4.00	4.00	4.00
SM frequency	GHz	1.58	1.63	1.60
Duration	μ s	383.26	600.22	491.74
Memory throughput	%	37.52	50.70	44.11
DRAM throughput	%	23.99	25.71	24.85
L1/TEX throughput	%	38.02	52.34	45.18
L2 throughput	%	37.68	45.71	41.69
SM throughput	%	43.80	57.55	50.68

Table 8 reports the corresponding launch and occupancy statistics. These numbers show that the GEMM kernels are resource-limited: each CTA uses 168 registers per thread and approximately 201 KB of dynamic shared memory, allowing only one active block per SM.

Despite low achieved occupancy, the GEMM kernels maintain high active-thread efficiency and dominate the iteration because of the large amount of matrix-multiply work. The remaining headroom is therefore less about launch overhead and more about reducing resource pressure and data movement around the GEMM pipeline.

A.3.2 pull_scatter_bf16 Kernel

The pull_scatter kernel accounts for 81.31 μ s, or approximately 6.4% of the iteration. Table 9 shows that

Table 8. Launch and occupancy summary for CUTLASS FP8 grouped GEMM.

Metric	Unit	Value
Block size	threads	384
Grid size	CTAs	148
Waves per SM	–	1.00
Registers per thread	reg/thread	168
Dynamic shared memory per block	KB/block	201.22
Theoretical occupancy	%	18.75
Achieved occupancy	%	14.09
Achieved active warps per SM	warps	9.02
Warp cycles per issued instruction	cycles	5.46
Average active threads per warp	–	31.85

it reaches 62.92% of peak sustained DRAM throughput, making it the largest non-GEMM DRAM consumer in this workload.

Table 9. NCU throughput summary for pull_scatter_bf16.

Metric	Unit	Value
DRAM frequency	GHz	3.99
SM frequency	GHz	1.94
Duration	μ s	81.31
Memory throughput	%	62.92
DRAM throughput	%	62.92
L1/TEX throughput	%	33.87
L2 throughput	%	30.90
SM throughput	%	23.52
Achieved occupancy	%	43.50
Registers per thread	reg/thread	61
Grid size	CTAs	14107
Waves per SM	–	23.83

Unlike the GEMM kernels, pull_scatter launches many CTAs and reaches much higher occupancy. Its bottleneck is DRAM bandwidth: the kernel performs substantial output movement after GEMM2, and therefore becomes the main remaining non-GEMM optimization target. This supports the main text’s conclusion that fusing pull_scatter into the GEMM2 epilogue is the most promising next optimization for long-sequence workloads.

A.4 Summary

The NCU profile confirms the performance breakdown reported in the main text. On the slowest long-sequence FuseMoE workload, most time is spent in the two CUTLASS FP8 grouped GEMMs, while the largest non-GEMM cost is `pull_scatter`. The GEMMs are resource-constrained but already highly optimized, whereas `pull_scatter` remains DRAM-bandwidth dominated. Therefore, the most meaningful remaining optimization is not another standalone GEMM improvement, but reducing producer-consumer data movement by fusing scatter behavior into the GEMM2 epilogue.